

类型	缓存什么	被缓存在何处	延迟（周期数）	由谁管理
CPU寄存器	4字节或8字节字	芯片上的CPU寄存器	0	编译器
TLB	地址翻译	芯片上的TLB	0	硬件MMU
L1高速缓存	64字节块	芯片上的L1高速缓存	4	硬件
L2高速缓存	64字节块	芯片上的L2高速缓存	10	硬件
L3高速缓存	64字节块	芯片上的L3高速缓存	50	硬件
虚拟内存	4KB页	主存	200	硬件 + OS
缓冲区缓存	部分文件	主存	200	OS
磁盘缓存	磁盘扇区	磁盘控制器	100 000	控制器固件
网络缓存	部分文件	本地磁盘	10 000 000	NFS客户
浏览器缓存	Web页	本地磁盘	10 000 000	Web浏览器
Web缓存	Web页	远程服务器磁盘	1 000 000 000	Web代理服务器

图 6-23 缓存在现代计算机系统中无处不在。TLB：翻译后备缓冲器(Translation Lookaside Buffer)；MMU：内存管理单元(Memory Management Unit)；OS：操作系统(Operating System)；AFS：安德鲁文件系统(Andrew File System)；NFS：网络文件系统(Network File System)

6.4 高速缓存存储器

早期计算机系统的存储器层次结构只有三层：CPU 寄存器、DRAM 主存储器和磁盘存储。不过，由于 CPU 和主存之间逐渐增大的差距，系统设计者被迫在 CPU 寄存器文件和主存之间插入了一个小的 SRAM 高速缓存存储器，称为 L1 高速缓存(一级缓存)，如图 6-24 所示。L1 高速缓存的访问速度几乎和寄存器一样快，典型地是大约 4 个时钟周期。

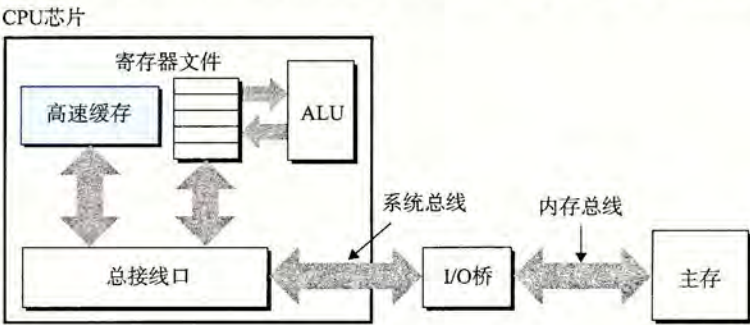


图 6-24 高速缓存存储器的典型总线结构

随着 CPU 和主存之间的性能差距不断增大，系统设计者在 L1 高速缓存和主存之间又插入了一个更大的高速缓存，称为 L2 高速缓存，可以在大约 10 个时钟周期内访问到它。有些现代系统还包括有一个更大的高速缓存，称为 L3 高速缓存，在存储器层次结构中，它位于 L2 高速缓存和主存之间，可以在大约 50 个周期内访问到它。虽然安排上有相当多的变化，但是通用原则是一样的。对于下一节中的讨论，我们会假设一个简单的存储器层次结构，CPU 和主存之间只有一个 L1 高速缓存。

6.4.1 通用的高速缓存存储器组织结构

考虑一个计算机系统，其中每个存储器地址有 m 位，形成 $M=2^m$ 个不同的地址。如图 6-25a 所示，这样一个机器的高速缓存被组织成一个有 $S=2^s$ 个高速缓存组(cache set)的

数组。每个组包含 E 个高速缓存行(cache line)。每个行是由一个 $B=2^b$ 字节的数据块(block)组成的, 一个有效位(valid bit)指明这个行是否包含有意义的信息, 还有 $t=m-(b+s)$ 个标记位(tag bit)(是当前块的内存地址的位的一个子集), 它们唯一地标识存储在这个高速缓存行中的块。

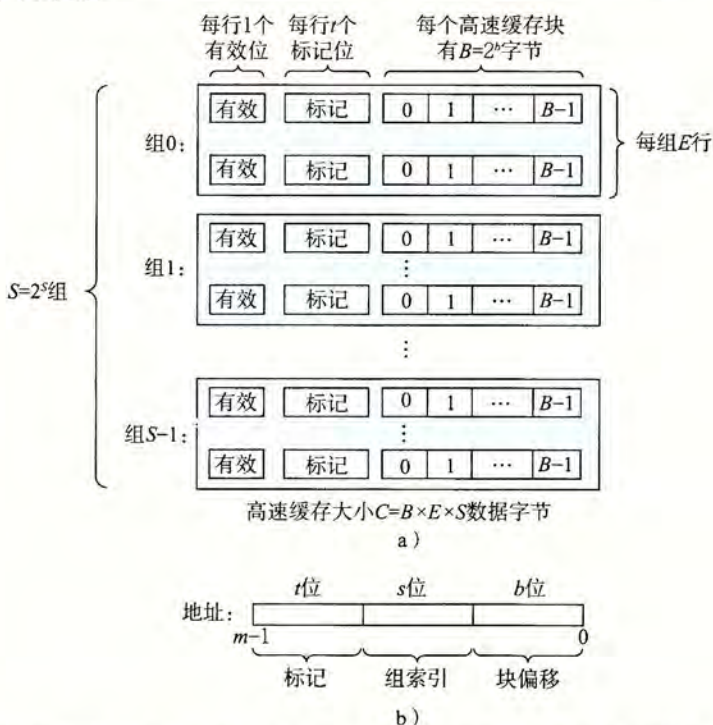


图 6-25 高速缓存 (S, E, B, m) 的通用组织。a) 高速缓存是一个高速缓存组的数组。每个组包含一个或多个行, 每个行包含一个有效位, 一些标记位, 以及一个数据块; b) 高速缓存的结构将 m 个地址位划分成了 t 个标记位、 s 个组索引位和 b 个块偏移位

一般而言, 高速缓存的结构可以用元组 (S, E, B, m) 来描述。高速缓存的大小(或容量) C 指的是所有块的大小的和。标记位和有效位不包括在内。因此, $C = S \times E \times B$ 。

当一条加载指令指示 CPU 从主存地址 A 中读一个字时, 它将地址 A 发送到高速缓存。如果高速缓存正保存着地址 A 处那个字的副本, 它就立即将那个字发回给 CPU。那么高速缓存如何知道它是否包含地址 A 处那个字的副本的呢? 高速缓存的结构使得它能够通过简单地检查地址位, 找到所请求的字, 类似于使用极其简单的哈希函数的哈希表。下面介绍它是如何工作的:

参数 S 和 B 将 m 个地址位分为了三个字段, 如图 6-25b 所示。 A 中 s 个组索引位是一个到 S 个组的数组的索引。第一个组是组 0, 第二个组是组 1, 依此类推。组索引位被解释为一个无符号整数, 它告诉我们这个字必须存储在哪个组中。一旦我们知道了这个字必须放在哪个组中, A 中的 t 个标记位就告诉我们这个组中的哪一行包含这个字(如果有的话)。当且仅当设置了有效位并且该行的标记位与地址 A 中的标记位相匹配时, 组中的这一行才包含这个字。一旦我们在由组索引标识的组中定位了由标号所标识的行, 那么 b 个块偏移位给出了在 B 个字节的数据块中的字偏移。

你可能已经注意到了, 对高速缓存的描述使用了很多符号。图 6-26 对这些符号做了个小结, 供你参考。

基 本 参 数	
参数	描述
$S=2^s$	组数
E	每个组的行数
$B=2^b$	块大小（字节）
$m=\log_2(M)$	（主存）物理地址位数

衍生出来的量	
参数	描述
$M=2^m$	内存地址的最大数量
$s=\log_2(S)$	组索引位数
$b=\log_2(B)$	块偏移位数
$t=m-(s+b)$	标记位数
$C=B \times E \times S$	不包括像有效位和标记位这样开销的高速缓存大小（字节）

图 6-26 高速缓存参数小结

 **练习题 6.9** 下表给出了几个不同的高速缓存的参数。确定每个高速缓存的高速缓存组数(S)、标记位数(t)、组索引位数(s)以及块偏移位数(b)。

高速缓存	m	C	B	E	S	t	s	b
1.	32	1024	4	1				
2.	32	1024	8	4				
3.	32	1024	32	32				

6.4.2 直接映射高速缓存

根据每个组的高速缓存行数 E ，高速缓存被分为不同的类。每个组只有一行($E=1$)的高速缓存称为直接映射高速缓存(direct-mapped cache)(见图 6-27)。直接映射高速缓存是最容易实现和理解的，所以我们会以它为例来说明一些高速缓存工作方式的通用概念。

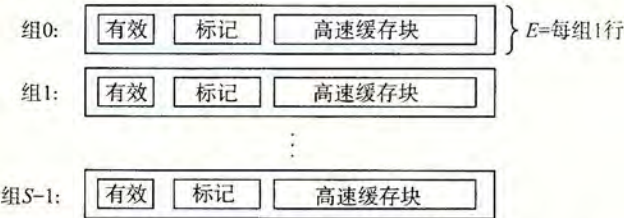


图 6-27 直接映射高速缓存($E=1$)。每个组只有一行

假设我们有这样一个系统，它有一个 CPU、一个寄存器文件、一个 L1 高速缓存和一个主存。当 CPU 执行一条读内存字 w 的指令，它向 L1 高速缓存请求这个字。如果 L1 高速缓存有 w 的一个缓存的副本，那么就得到 L1 高速缓存命中，高速缓存会很快抽取出 w ，并将它返回给 CPU。否则就是缓存不命中，当 L1 高速缓存向主存请求包含 w 的块的一个副本时，CPU 必须等待。当被请求的块最终从内存到达时，L1 高速缓存将这个块存放在它的一个高速缓存行里，从被存储的块中抽取出字 w ，然后将它返回给 CPU。高速

缓存确定一个请求是否命中，然后抽取出被请求的字的的过程，分为三步：1) 组选择；2) 行匹配；3) 字抽取。

1. 直接映射高速缓存中的组选择

在这一步中，高速缓存从 w 的地址中间抽取出 s 个组索引位。这些位被解释成一个对应于一个组号的无符号整数。换句话说，如果我们把高速缓存看成是一个关于组的一维数组，那么这些组索引位就是一个到这个数组的索引。图 6-28 展示了直接映射高速缓存的组选择是如何工作的。在这个例子中，组索引位 00001_2 被解释为一个选择组 1 的整数索引。

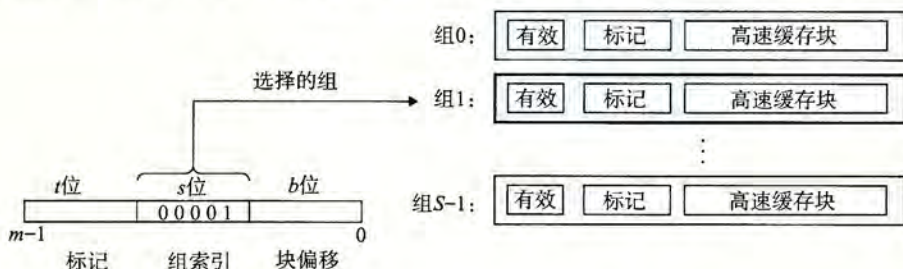


图 6-28 直接映射高速缓存中的组选择

2. 直接映射高速缓存中的行匹配

在上一步中我们已经选择了某个组 i ，接下来的一步就要确定是否有字 w 的一个副本存储在组 i 包含的一个高速缓存行中。在直接映射高速缓存中这很容易，而且很快，这是因为每个组只有一行。当且仅当设置了有效位，而且高速缓存行中的标记与 w 的地址中的标记相匹配时，这一行中包含 w 的一个副本。

图 6-29 展示了直接映射高速缓存中行匹配是如何工作的。在这个例子中，选中的组中只有一个高速缓存行。这个行的有效位设置了，所以我们知道标记和块中的位是有意义的。因为这个高速缓存行中的标记位与地址中的标记位相匹配，所以我们知道我们想要的那个字的一个副本确实存储在这个行中。换句话说，我们得到一个缓存命中。另一方面，如果有效位没有设置，或者标记不相匹配，那么我们就得到一个缓存不命中。

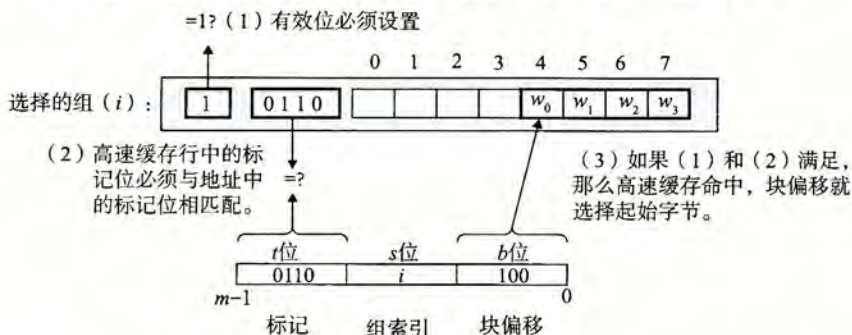


图 6-29 直接映射高速缓存中的行匹配和字选择。在高速缓存块中， w_0 表示字 w 的低位字节， w_1 是下一个字节，依此类推

3. 直接映射高速缓存中的字选择

一旦命中，我们知道 w 就在这个块中的某个地方。最后一步确定所需要的字在块是从哪里开始的。如图 6-29 所示，块偏移位提供了所需要的字的第一个字节的偏移。就像我们把高速缓存看成一个行的数组一样，我们把块看成一个字节的数组，而字节偏移是到

这个数组的一个索引。在这个示例中，块偏移位是 100_2 ，它表明 w 的副本是从块中的字节 4 开始的(我们假设字长为 4 字节)。

4. 直接映射高速缓存中不命中时的行替换

如果缓存不命中，那么它需要从存储器层次结构中的下一层取出被请求的块，然后将新的块存储在组索引位指示的组中的一个高速缓存行中。一般而言，如果组中都是有效高速缓存行了，那么必须要驱逐出一个现存的行。对于直接映射高速缓存来说，每个组只包含有一行，替换策略非常简单：用新取出的行替换当前的行。

5. 综合：运行中的直接映射高速缓存

高速缓存用来选择组和标识行的机制极其简单，因为硬件必须在几个纳秒的时间内完成这些工作。不过，用这种方式来处理位是很令人困惑的。一个具体的例子能帮助解释清楚这个过程。假设我们有一个直接映射高速缓存，描述如下

$(S,E,B,m) = (4,1,2,4)$

换句话说，高速缓存有 4 个组，每个组一行，每个块 2 个字节，而地址是 4 位的。我们还假设每个字都是单字节的。当然，这样一些假设完全是不现实的，但是它们能使示例保持简单。

当你初学高速缓存时，列举出整个地址空间并划分好位是很有帮助的，就像我们在图 6-30 对 4 位的示例所做的那样。关于这个列举出的空间，有一些有趣的事情值得注意：

地址 (十进制)	地址位			块号 (十进制)
	标记位 ($t=1$)	索引位 ($s=2$)	偏移位 ($b=1$)	
0	0	00	0	0
1	0	00	1	0
2	0	01	0	1
3	0	01	1	1
4	0	10	0	2
5	0	10	1	2
6	0	11	0	3
7	0	11	1	3
8	1	00	0	4
9	1	00	1	4
10	1	01	0	5
11	1	01	1	5
12	1	10	0	6
13	1	10	1	6
14	1	11	0	7
15	1	11	1	7

图 6-30 示例直接映射高速缓存的 4 位地址空间

- 标记位和索引位连起来唯一地标识了内存中的每个块。例如，块 0 是由地址 0 和 1 组成的，块 1 是由地址 2 和 3 组成的，块 2 是由地址 4 和 5 组成的，依此类推。
 - 因为有 8 个内存块，但是只有 4 个高速缓存组，所以多个块会映射到同一个高速缓存组(即它们有相同的组索引)。例如，块 0 和 4 都映射到组 0，块 1 和 5 都映射到组 1，等等。
 - 映射到同一个高速缓存组的块由标记位唯一地标识。例如，块 0 的标记位为 0，而块 4 的标记位为 1，块 1 的标记位为 0，而块 5 的标记位为 1，依此类推。
- 让我们来模拟一下当 CPU 执行一系列读的时候，高速缓存的执行情况。记住对于这

个示例，我们假设 CPU 读 1 字节的字。虽然这种手工的模拟很乏味，你可能想要跳过它，但是根据我们的经验，在学生们做过几个这样的练习之前，他们是不能真正理解高速缓存是如何工作的。

初始时，高速缓存是空的(即每个有效位都是 0)：

组	有效位	标记位	块[0]	块[1]
0	0			
1	0			
2	0			
3	0			

表中的每一行都代表一个高速缓存行。第一列表明该行所属的组，但是请记住提供这个位只是为了方便，实际上它并不真是高速缓存的一部分。后面四列代表每个高速缓存行的实际的位。现在，让我们来看看当 CPU 执行一系列读时，都发生了什么：

1) 读地址 0 的字。因为组 0 的有效位是 0，是缓存不命中。高速缓存从内存(或低一层的高速缓存)取出块 0，并把这个块存储在组 0 中。然后，高速缓存返回新取出的高速缓存行的块[0]的 $m[0]$ (内存位置 0 的内容)。

组	有效位	标记位	块[0]	块[1]
0	1	0	$m[0]$	$m[1]$
1	0			
2	0			
3	0			

2) 读地址 1 的字。这次会是高速缓存命中。高速缓存立即从高速缓存行的块[1]中返回 $m[1]$ 。高速缓存的状态没有变化。

3) 读地址 13 的字。由于组 2 中的高速缓存行不是有效的，所以有缓存不命中。高速缓存把块 6 加载到组 2 中，然后从新的高速缓存行的块[1]中返回 $m[13]$ 。

组	有效位	标记位	块[0]	块[1]
0	1	0	$m[0]$	$m[1]$
1	0			
2	1	1	$m[12]$	$m[13]$
3	0			

4) 读地址 8 的字。这会发生缓存不命中。组 0 中的高速缓存行确实是有效的，但是标记不匹配。高速缓存将块 4 加载到组 0 中(替换读地址 0 时读入的那一行)，然后从新的高速缓存行的块[0]中返回 $m[8]$ 。

组	有效位	标记位	块[0]	块[1]
0	1	1	$m[8]$	$m[9]$
1	0			
2	1	1	$m[12]$	$m[13]$
3	0			

5) 读地址 0 的字。又会发生缓存不命中，因为在前面引用地址 8 时，我们刚好替换了块 0。这就是冲突不命中的一个例子，也就是我们有足够的高速缓存空间，但是却交替地引用映射到同一个组的块。

组	有效位	标记位	块[0]	块[1]
0	1	0	m[0]	m[1]
1	0			
2	1	1	m[12]	m[13]
3	0			

6. 直接映射高速缓存中的冲突不命中

冲突不命中在真实的程序中很常见，会导致令人困惑的性能问题。当程序访问大小为 2 的幂的数组时，直接映射高速缓存中通常会发生冲突不命中。例如，考虑一个计算两个向量点积的函数：

```
1 float dotprod(float x[8], float y[8])
2 {
3     float sum = 0.0;
4     int i;
5
6     for (i = 0; i < 8; i++)
7         sum += x[i] * y[i];
8     return sum;
9 }
```

对于 x 和 y 来说，这个函数有良好的空间局部性，因此我们期望它的命中率会比较高。不幸的是，并不总是如此。

假设浮点数是 4 个字节，x 被加载到从地址 0 开始的 32 字节连续内存中，而 y 紧跟在 x 之后，从地址 32 开始。为了简便，假设一个块是 16 个字节（足够容纳 4 个浮点数），高速缓存由两个组组成，高速缓存的整个大小为 32 字节。我们会假设变量 sum 实际上存放在一个 CPU 寄存器中，因此不需要内存引用。根据这些假设每个 x[i] 和 y[i] 会映射到相同的高速缓存组：

元素	地址	组索引	元素	地址	组索引
x[0]	0	0	y[0]	32	0
x[1]	4	0	y[1]	36	0
x[2]	8	0	y[2]	40	0
x[3]	12	0	y[3]	44	0
x[4]	16	1	y[4]	48	1
x[5]	20	1	y[5]	52	1
x[6]	24	1	y[6]	56	1
x[7]	28	1	y[7]	60	1

在运行时，循环的第一次迭代引用 x[0]，缓存不命中会导致包含 x[0]~x[3] 的块被加载到组 0。接下来是对 y[0] 的引用，又一次缓存不命中，导致包含 y[0]~y[3] 的块被复制到组 0，覆盖前一次引用复制进来的 x 的值。在下一次迭代中，对 x[1] 的引用不命中，导致 x[0]~x[3] 的块被加载回组 0，覆盖掉 y[0]~y[3] 的块。因而现在我们就有了一个冲突不命中，而且实际上后面每次对 x 和 y 的引用都会导致冲突不命中，因为我们在 x 和 y 的块之间抖动 (thrash)。术语“抖动”描述的是这样一种情况，即高速缓存反复地加载和驱逐相同的高速缓存块的组。

简要说来就是，即使程序有良好的空间局部性，而且我们的高速缓存中也有足够的空间来存放 x[i] 和 y[i] 的块，每次引用还是会导致冲突不命中，这是因为这些块被映射到了同

一个高速缓存组。这种抖动导致速度下降 2 或 3 倍并不稀奇。另外，还要注意虽然我们的示例极其简单，但是对于更大、更现实的直接映射高速缓存来说，这个问题也是很真实的。

幸运的是，一旦程序员意识到了正在发生什么，就很容易修正抖动问题。一个很简单的方法是在每个数组的结尾放 B 字节的填充。例如，不是将 x 定义为 `float x[8]`，而是定义成 `float x[12]`。假设在内存中 y 紧跟在 x 后面，我们有下面这样的从数组元素到组的映射：

元素	地址	组索引	元素	地址	组索引
$x[0]$	0	0	$y[0]$	48	1
$x[1]$	4	0	$y[1]$	52	1
$x[2]$	8	0	$y[2]$	56	1
$x[3]$	12	0	$y[3]$	60	1
$x[4]$	16	1	$y[4]$	64	0
$x[5]$	20	1	$y[5]$	68	0
$x[6]$	24	1	$y[6]$	72	0
$x[7]$	28	1	$y[7]$	76	0

在 x 结尾加了填充， $x[i]$ 和 $y[i]$ 现在就映射到了不同的组，消除了抖动冲突不命中。

 **练习题 6.10** 在前面 `dotprod` 的例子中，在我们对数组 x 做了填充之后，所有对 x 和 y 的引用的命中率是多少？

旁注 为什么用中间的位来做索引

你也许会奇怪，为什么高速缓存用中间的位来作为组索引，而不是用高位。为什么用中间的位更好，是有很好的原因的。图 6-31 说明了原因。如果高位用做索引，那么一些连续的内存块就会映射到相同的高速缓存块。例如，在图中，头四个块映射到第一个高速缓存组，第二个四个块映射到第二个组，依此类推。如果一个程序有良好的空间局部性，顺序扫描一个数组的元素，那么在任意时刻，高速缓存都只保存着一个块大小

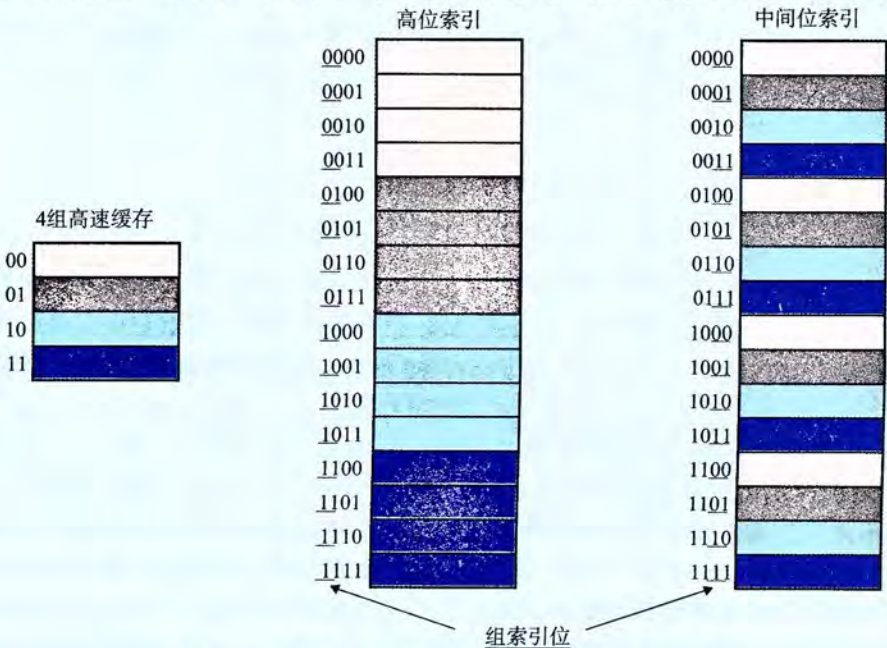


图 6-31 为什么用中间位来作为高速缓存的索引

的数组内容。这样对高速缓存的使用效率很低。相比较而言,以中间位作为索引,相邻的块总是映射到不同的高速缓存行。在这里的情况中,高速缓存能够存放整个大小为 C 的数组片,这里 C 是高速缓存的大小。

 **练习题 6.11** 假想一个高速缓存,用地址的高 s 位做组索引,那么内存块连续的片(chunk)会被映射到同一个高速缓存组。

A. 每个这样的连续的数组片中有多少个块?

B. 考虑下面的代码,它运行在一个高速缓存形式为 $(S, E, B, m) = (512, 1, 32, 32)$ 的系统上:

```
int array[4096];

for (i = 0; i < 4096; i++)
    sum += array[i];
```

在任意时刻,存储在高速缓存中的数组块的最大数量为多少?

6.4.3 组相联高速缓存

直接映射高速缓存中冲突不命中造成的问题源于每个组只有一行(或者,按照我们的术语来描述就是 $E=1$)这个限制。组相联高速缓存(set associative cache)放松了这条限制,所以每个组都保存有多于一个的高速缓存行。一个 $1 < E < C/B$ 的高速缓存通常称为 E 路组相联高速缓存。在下一节中,我们会讨论 $E=C/B$ 这种特殊情况。图 6-32 展示了一个 2 路组相联高速缓存的结构。



图 6-32 组相联高速缓存($1 < E < C/B$)。在一个组相联高速缓存中,每个组包含多于一个行。
这里的特例是一个 2 路组相联高速缓存

1. 组相联高速缓存中的组选择

它的组选择与直接映射高速缓存的组选择一样,组索引位标识组。图 6-33 总结了这个原理。

2. 组相联高速缓存中的行匹配和字选择

组相联高速缓存中的行匹配比直接映射高速缓存中的更复杂,因为它必须检查多个行的标记位和有效位,以确定所请求的字是否在集合中。传统的内存是一个值的数组,以地址作为输入,并返回存储在那个地址的值。另一方面,相联存储器是一个(key, value)对的数组,以 key 为输入,返回与输入的 key 相匹配的(key, value)对中的 value 值。因此,我们可以把组相联高速缓存中的每个组都看成一个小的相联存储器, key 是标记和有效位,而 value 就是块的内容。

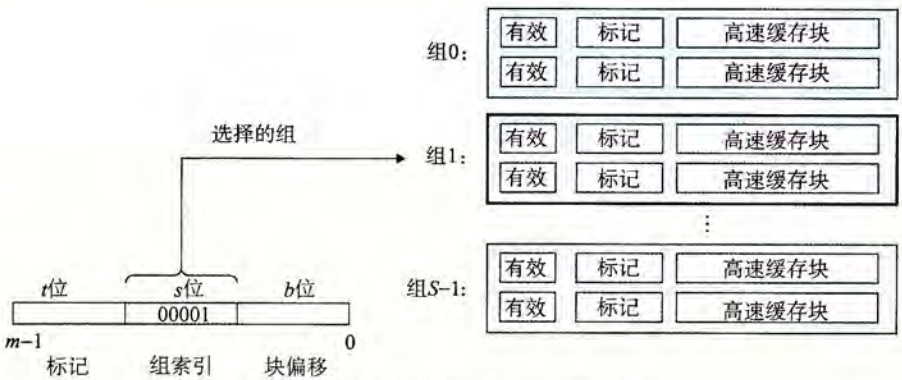


图 6-33 组相联高速缓存中的组选择

图 6-34 展示了相联高速缓存中行匹配的基本思想。这里的一个重要思想就是组中的任何一行都可以包含任何映射到这个组的内存块。所以高速缓存必须搜索组中的每一行，寻找一个有效的行，其标记与地址中的标记相匹配。如果高速缓存找到了这样一行，那么我们就命中，块偏移从这个块中选择一个字，和前面一样。

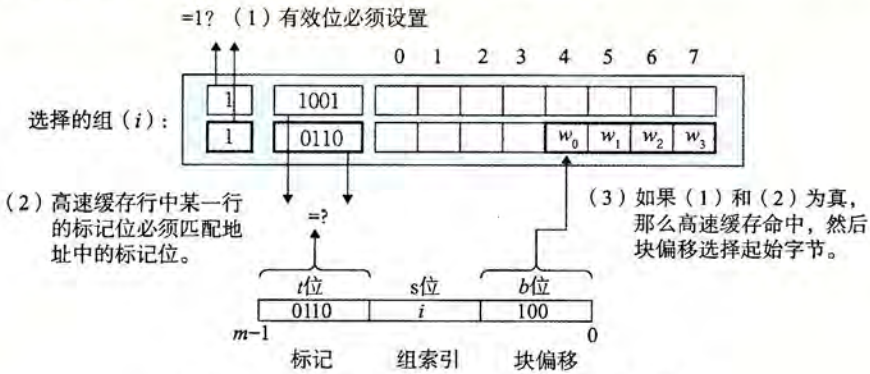


图 6-34 组相联高速缓存中的行匹配和字选择

3. 组相联高速缓存中不命中时的行替换

如果 CPU 请求的字不在组的任何一行中，那么就是缓存不命中，高速缓存必须从内存中取出包含这个字的块。不过，一旦高速缓存取出了这个块，该替换哪个行呢？当然，如果有一个空行，那它就是个很好的候选。但是如果该组中没有空行，那么我们必须从中选择一个非空的行，希望 CPU 不会很快引用这个被替换的行。

程序员很难在代码中利用高速缓存替换策略，所以在此我们不会过多地讲述其细节。最简单的替换策略是随机选择要替换的行。其他更复杂的策略利用了局部性原理，以使在比较近的将来引用被替换的行的概率最小。例如，最不常使用 (Least-Frequently-Used, LFU) 策略会替换在过去某个时间窗口内引用次数最少的那一行。最近最少使用 (Least-Recently-Used, LRU) 策略会替换最后一次访问时间最久远的那一行。所有这些策略都需要额外的时间和硬件。但是，越往存储器层次结构下面走，远离 CPU，一次不命中的开销就会更加昂贵，用更好的替换策略使得不命中最少也变得更加值得了。

6.4.4 全相联高速缓存

全相联高速缓存 (fully associative cache) 是由一个包含所有高速缓存行的组 (即 $E=C/$

B)组成的。图 6-35 给出了基本结构。

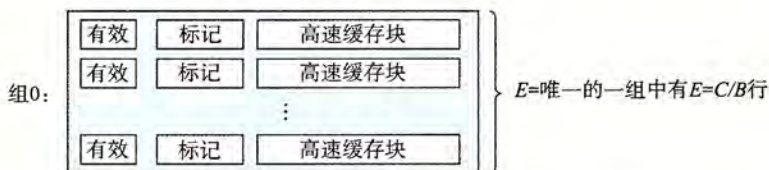


图 6-35 全相联高速缓存($E=C/B$)。在全相联高速缓存中, 一个组包含所有的行

1. 全相联高速缓存中的组选择

全相联高速缓存中的组选择非常简单, 因为只有一个组, 图 6-36 做了个小结。注意地址中没有组索引位, 地址只被划分成了一个标记和一个块偏移。

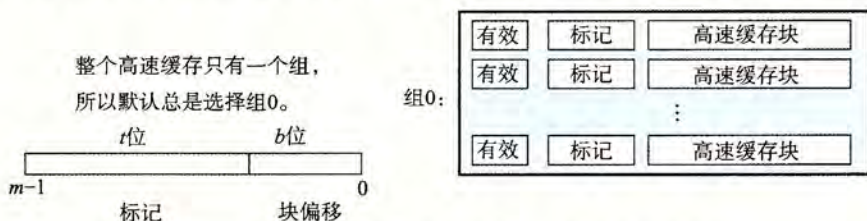


图 6-36 全相联高速缓存中的组选择。注意没有组索引位

2. 全相联高速缓存中的行匹配和字选择

全相联高速缓存中的行匹配和字选择与组相联高速缓存中的是一样的, 如图 6-37 所示。它们之间的区别主要是规模大小的问题。

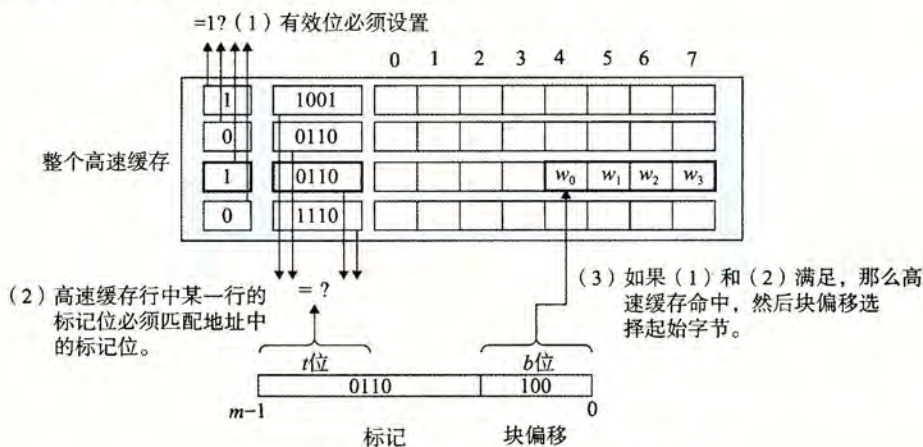


图 6-37 全相联高速缓存中的行匹配和字选择

因为高速缓存电路必须并行地搜索许多相匹配的标记, 构造一个又大又快相联高速缓存很困难, 而且很昂贵。因此, 全相联高速缓存只适合做小的高速缓存, 例如虚拟内存系统中的翻译备用缓冲器(TLB), 它缓存页表项(见 9.6.2 节)。

练习题 6.12 下面的问题能帮助你加强理解高速缓存是如何工作的。有如下假设:

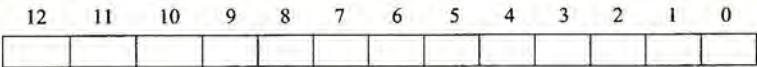
- 内存是字节寻址的。
- 内存访问的是 1 字节的字(不是 4 字节的字)。

- 地址的宽度为 13 位。
 - 高速缓存是 2 路组相联的($E=2$)，块大小为 4 字节($B=4$)，有 8 个组($S=8$)。
- 高速缓存的内容如下，所有的数字都是以十六进制来表示的：

2路组相联高速缓存												
	行0						行1					
组索引	标记位	有效位	字节0	字节1	字节2	字节3	标记位	有效位	字节0	字节1	字节2	字节3
0	09	1	86	30	3F	10	00	0	—	—	—	—
1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
2	EB	0	—	—	—	—	0B	0	—	—	—	—
3	06	0	—	—	—	—	32	1	12	08	7B	AD
4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
5	71	1	0B	DE	18	4B	6E	0	—	—	—	—
6	91	1	A0	B7	26	2D	F0	0	—	—	—	—
7	46	0	—	—	—	—	DE	1	12	C0	88	37

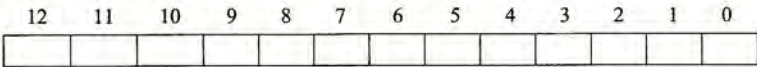
下面的图展示的是地址格式(每个小方框一个位)。指出(在图中标出)用来确定下列内容的字段：

- CO 高速缓存块偏移
- CI 高速缓存组索引
- CT 高速缓存标记



 **练习题 6.13** 假设一个程序运行在练习题 6-12 中的机器上，它引用地址 0x0E34 处的 1 个字节的字。指出访问的高速缓存条目和十六进制表示的返回的高速缓存字节值。指出是否会发生缓存不命中。如果会出现缓存不命中，用 “—” 来表示 “返回的高速缓存字节”。

A. 地址格式(每个小方框一个位)：

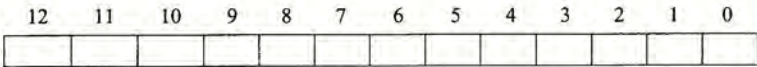


B. 内存引用：

参数	值
高速缓存块偏移 (CO)	0x_____
高速缓存组索引 (CI)	0x_____
高速缓存标记 (CT)	0x_____
高速缓存命中？ (是/否)	
返回的高速缓存字节	0x_____

 **练习题 6.14** 对于存储器地址 0x0DD5，再做一遍练习题 6.13。

A. 地址格式(每个小方框一个位)：



B. 内存引用：

参数	值
高速缓存块偏移 (CO)	0x_____
高速缓存组索引 (CI)	0x_____
高速缓存标记 (CT)	0x_____
高速缓存命中? (是/否)	
返回的高速缓存字节	0x_____

 **练习题 6.15** 对于内存地址 0x1FE4，再做一遍练习题 6.13。

A. 地址格式(每个小方框一个位):

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 内存引用:

参数	值
高速缓存块偏移 (CO)	0x_____
高速缓存组索引 (CI)	0x_____
高速缓存标记 (CT)	0x_____
高速缓存命中? (是/否)	
返回的高速缓存字节	0x_____

 **练习题 6.16** 对于练习题 6.12 中的高速缓存，列出所有的在组 3 中会命中的十六进制内存地址。

6.4.5 有关写的问题

正如我们看到的，高速缓存关于读的操作非常简单。首先，在高速缓存中查找所需字 w 的副本。如果命中，立即返回字 w 给 CPU。如果不命中，从存储器层次结构中较低层中取出包含字 w 的块，将这个块存储到某个高速缓存行中(可能会驱逐一个有效的行)，然后返回字 w 。

写的情况就要复杂一些了。假设我们要写一个已经缓存了的字 w (写命中，write hit)。在高速缓存更新了它的 w 的副本之后，怎么更新 w 在层次结构中紧接着低一层中的副本呢？最简单的方法，称为直写(write-through)，就是立即将 w 的高速缓存块写回到紧接着的低一层中。虽然简单，但是直写的缺点是每次写都会引起总线流量。另一种方法，称为写回(write-back)，尽可能地推迟更新，只有当替换算法要驱逐这个更新过的块时，才把它写到紧接着的低一层中。由于局部性，写回能显著地减少总线流量，但是它的缺点是增加了复杂性。高速缓存必须为每个高速缓存行维护一个额外的修改位(dirty bit)，表明这个高速缓存块是否被修改过。

另一个问题是如何处理写不命中。一种方法，称为写分配(write-allocate)，加载相应的低一层中的块到高速缓存中，然后更新这个高速缓存块。写分配试图利用写的空间局部性，但是缺点是每次不命中都会导致一个块从低一层传送到高速缓存。另一种方法，称为非写分配(not-write-allocate)，避开高速缓存，直接把这个字写到低一层中。直写高速缓存通常是非写分配的。写回高速缓存通常是写分配的。

为写操作优化高速缓存是一个细致而困难的问题，在此我们只略讲皮毛。细节随系统的不同而不同，而且通常是私有的，文档记录不详细。对于试图编写高速缓存比较友好的

程序的程序员来说,我们建议在心里采用一个使用写回和写分配的高速缓存的模型。这样建议有几个原因。通常,由于较长的传送时间,存储器层次结构中较低层的缓存更可能使用写回,而不是直写。例如,虚拟内存系统(用主存作为存储在磁盘上的块的缓存)只使用写回。但是由于逻辑电路密度的提高,写回的高复杂性也越来越不成为阻碍了,我们在现代系统的所有层次上都能看到写回缓存。所以这种假设符合当前的趋势。假设使用写回写分配方法的另一个原因是,它与处理读的方式相对称,因为写回写分配试图利用局部性。因此,我们可以在高层次上开发我们的程序,展示良好的空间和时间局部性,而不是试图为某一个存储器系统进行优化。

6.4.6 一个真实的高速缓存层次结构的解剖

到目前为止,我们一直假设高速缓存只保存程序数据。不过,实际上,高速缓存既保存数据,也保存指令。只保存指令的高速缓存称为 *i-cache*。只保存程序数据的高速缓存称为 *d-cache*。既保存指令又包括数据的高速缓存称为统一的高速缓存(unified cache)。现代处理器包括独立的 *i-cache* 和 *d-cache*。这样做有很多原因。有两个独立的高速缓存,处理器能够同时读一个指令字和一个数据字。*i-cache* 通常是只读的,因此比较简单。通常会针对不同的访问模式来优化这两个高速缓存,它们可以有不同的块大小,相联度和容量。使用不同的高速缓存也确保了数据访问不会与指令访问形成冲突不命中,反过来也是一样,代价就是可能会引起容量不命中增加。

图 6-38 给出了 Intel Core i7 处理器的高速缓存层次结构。每个 CPU 芯片有四个核。每个核有自己私有的 L1 *i-cache*、L1 *d-cache* 和 L2 统一的高速缓存。所有的核共享片上 L3 统一的高速缓存。这个层次结构的一个有趣的特性是所有的 SRAM 高速缓存存储器都在 CPU 芯片上。

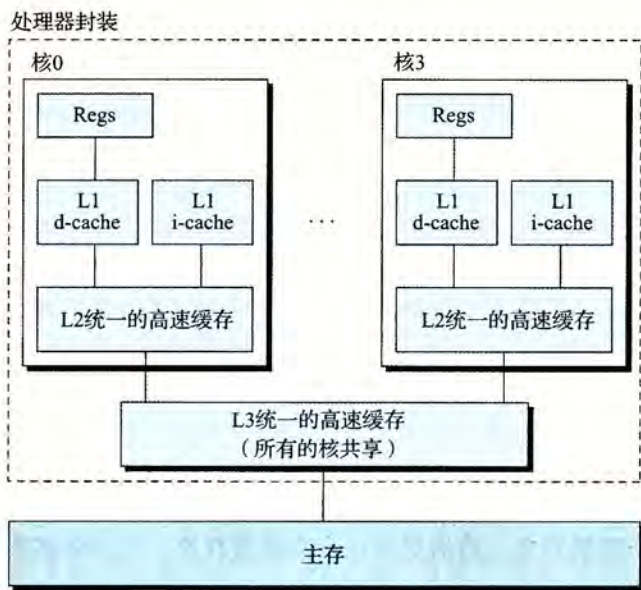


图 6-38 Intel Core i7 的高速缓存层次结构

图 6-39 总结了 Core i7 高速缓存的基本特性。

高速缓存类型	访问时间(周期)	高速缓存大小(C)	相联度(E)	块大小(B)	组数(S)
L1 i-cache	4	32KB	8	64B	64
L1 d-cache	4	32KB	8	64B	64
L2统一的高速缓存	10	256KB	8	64B	512
L3统一的高速缓存	40~75	8MB	16	64B	8192

图 6-39 Core i7 高速缓存层次结构的特性

6.4.7 高速缓存参数的性能影响

有许多指标来衡量高速缓存的性能:

- 不命中率(miss rate)。在一个程序执行或程序的一部分执行期间,内存引用不命中的比率。它是这样计算的:不命中数量/引用数量。
- 命中率(hit rate)。命中的内存引用比率。它等于 $1 - \text{不命中率}$ 。
- 命中时间(hit time)。从高速缓存传送一个字到 CPU 所需的时间,包括组选择、行确认和字选择的时间。对于 L1 高速缓存来说,命中时间的数量级是几个时钟周期。
- 不命中处罚(miss penalty)。由于不命中所需要的时间。L1 不命中需要从 L2 得到服务的处罚,通常是数 10 个周期;从 L3 得到服务的处罚,50 个周期;从主存得到的服务的处罚,200 个周期。

优化高速缓存的成本和性能的折中是一项很精细的工作,它需要在现实的基准程序代码上进行大量的模拟,因此超出了我们讨论的范围。不过,还是可以认识一些定性的折中考量的。

1. 高速缓存大小的影响

一方面,较大的高速缓存可能会提高命中率。另一方面,使大存储器运行得更快总是要难一些的。结果,较大的高速缓存可能会增加命中时间。这解释了为什么 L1 高速缓存比 L2 高速缓存小,以及为什么 L2 高速缓存比 L3 高速缓存小。

2. 块大小的影响

大的块有利有弊。一方面,较大的块能利用程序中可能存在的空间局部性,帮助提高命中率。不过,对于给定的高速缓存大小,块越大就意味着高速缓存行数越少,这会损害时间局部性比空间局部性更好的程序中的命中率。较大的块对不命中处罚也有负面影响,因为块越大,传送时间就越长。现代系统(如 Core i7)会折中使高速缓存块包含 64 个字节。

3. 相联度的影响

这里的问题是参数 E 选择的影响, E 是每个组中高速缓存行数。较高的相联度(也就是 E 的值较大)的优点是降低了高速缓存由于冲突不命中出现抖动的可能性。不过,较高的相联度会造成较高的成本。较高的相联度实现起来很昂贵,而且很难使之速度变快。每一行需要更多的标记位,每一行需要额外的 LRU 状态位和额外的控制逻辑。较高的相联度会增加命中时间,因为复杂性增加了,另外,还会增加不命中处罚,因为选择牺牲行的复杂性也增加了。

相联度的选择最终变成了命中时间和不命中处罚之间的折中。传统上,努力争取时钟频率的高性能系统会为 L1 高速缓存选择较低的相联度(这里的不命中处罚只是几个周期),而在不命中处罚比较高的较低层上使用比较小的相联度。例如,Intel Core i7 系统中,L1 和 L2 高速缓存是 8 路组相联的,而 L3 高速缓存是 16 路组相联的。

4. 写策略的影响

直写高速缓存比较容易实现,而且能使用独立于高速缓存的写缓冲区(write buffer),用来更新内存。此外,读不命中开销没这么大,因为它们不会触发内存写。另一方面,写

回高速缓存引起的传送比较少，它允许更多的到内存的带宽用于执行 DMA 的 I/O 设备。此外，越往层次结构下面走，传送时间增加，减少传送的数量就变得更加重要。一般而言，高速缓存越往下层，越可能使用写回而不是直写。

旁注 高速缓存行、组和块有什么区别？

很容易混淆高速缓存行、组和块之间的区别。让我们回顾一下这些概念，确保概念清晰：

- 块是一个固定大小的信息包，在高速缓存和主存(或下一层高速缓存)之间来回传送。
- 行是高速缓存中的一个容器，存储块以及其他信息(例如有效位和标记位)。
- 组是一个或多个行的集合。直接映射高速缓存中的组只由一行组成。组相联和全相联高速缓存中的组是由多个行组成的。

在直接映射高速缓存中，组和行实际上是等价的。不过，在相联高速缓存中，组和行是很不一样的，这两个词不能互换使用。

因为一行总是存储一个块，术语“行”和“块”通常互换使用。例如，系统专家总是说高速缓存的“行大小”，实际上他们指的是块大小。这样的用法十分普遍，只要你理解块和行之间的区别，它不会造成任何误会。

6.5 编写高速缓存友好的代码

在 6.2 节中，我们介绍了局部性的思想，而且定性地谈了一下什么会具有良好的局部性。明白了高速缓存存储器是如何工作的，我们就能更加准确一些了。局部性比较好的程序更容易有较低的不命中率，而不命中率较低的程序往往比不命中率较高的程序运行得更快。因此，从具有良好局部性的意义上来说，好的程序员总是应该试着去编写高速缓存友好(cache friendly)的代码。下面就是我们用来确保代码高速缓存友好的基本方法。

1) 让最常见的情况运行得快。程序通常把大部分时间都花在少量的核心函数上，而這些函数通常把大部分时间都花在了少量循环上。所以要把注意力集中在核心函数里的循环上，而忽略其他部分。

2) 尽量减小每个循环内部的缓存不命中数量。在其他条件(例如加载和存储的总次数)相同的情况下，不命中率较低的循环运行得更快。

为了看看实际上这是怎么工作的，考虑 6.2 节中的函数 `sumvec`：

```
1  int sumvec(int v[N])
2  {
3      int i, sum = 0;
4
5      for (i = 0; i < N; i++)
6          sum += v[i];
7      return sum;
8  }
```

这个函数高速缓存友好吗？首先，注意对于局部变量 `i` 和 `sum`，循环体有良好的时间局部性。实际上，因为它们都是局部变量，任何合理的优化编译器都会把它们缓存在寄存器文件中，也就是存储器层次结构的最高层中。现在考虑一下对向量 `v` 的步长为 1 的引用。一般而言，如果一个高速缓存的块大小为 B 字节，那么一个步长为 k 的引用模式(这里 k 是以字为单位的)平均每次循环迭代会有 $\min(1, (\text{wordsize} \times k) / B)$ 次缓存不命中。当 $k=1$ 时，它取最小值，所以对 `v` 的步长为 1 的引用确实是高速缓存友好的。例如，假设 `v` 是块对齐的，字为 4 个字节，高速缓存块为 4 个字，而高速缓存初始为空(冷高速缓存)。然